Exceptional Control Flow II

Outline

- Processes
- Context Switch
- Concurrency
- System Call Error Handling
- Exiting & Creating Processes
- Suggested reading 8.2, 8.3, 8.4.1, 8.4.2

Processes

When a New Process is Created

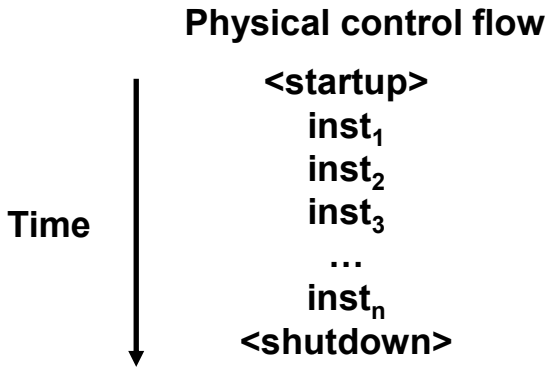
- Each time a user runs a program
  - by typing the name of an executable object file to the shell
    - the shell creates a new process
    - and then runs the executable object file
      - in the context of this new process
  - by clicking an icon of an application on the desk

When a New Process is Created

- Application programs can also
  - create new processes
  - and run
    - either their own
    - or other application
  - in the context of new process

Control flow

- From startup to shutdown, a CPU simply reads and executes a sequence of instructions, one at a time.
- This sequence is the system's physical *control flow* (or *flow of control*).



Processes

- Concurrency
  - Multiple processes can run concurrently
- How to support concurrency
  - Interleaving (multi-tasking)
    - The instructions of one process are interleaved with the instructions of another process
  - Virtual address (Chap. 9)
- How to implement interleaving
  - Context switch

Context Switches

Context

- The kernel maintains a context for each process
- The context is the state that the kernel needs to restart an interrupted process
- Context contains
  - the program's code and data stored in memory
  - Value of PC, register file, status registers
  - user's stack, kernel's stack
  - environment variables
  - Kernel data structures
    - Process table, page table, file table

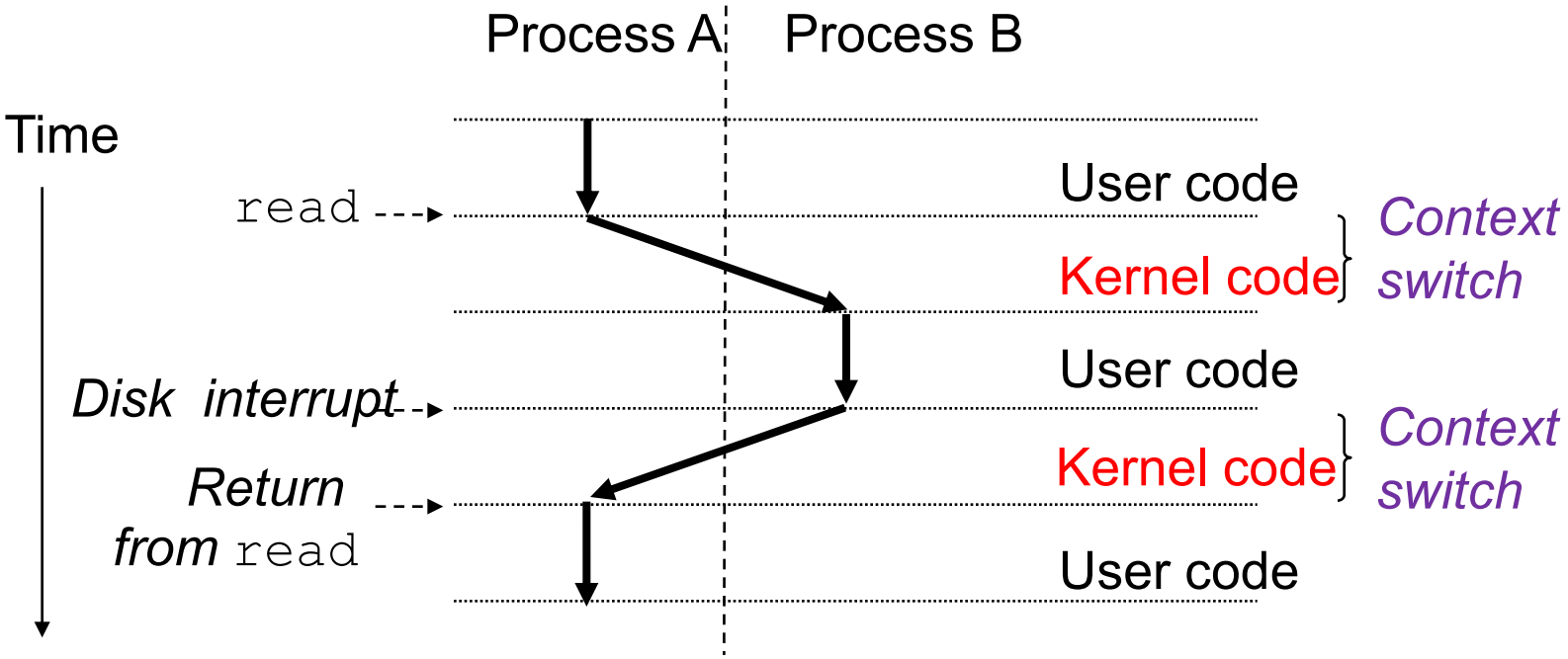
Context switching

- There are many processes in a computer system at one time
  - How can a computer manage these processes running
- The multitasking (time slicing)
  - A mechanism that the OS performs
    - The basic operation is context switch
  - Higher-level form of exceptional control flow
    - Built on top of the lower-level **exception** mechanism

Context switching

- When does the context switch happen?
  - The kernel is executing a system call on behalf of the user such as
    - `read`, `sleep` , `etc.` which will cause the calling process blocked
    - Even if a system call does not block, the kernel can decide to perform a context switch rather than return control to the calling process
  - As a result of an interrupt
    - Timer interrupt

Context switching



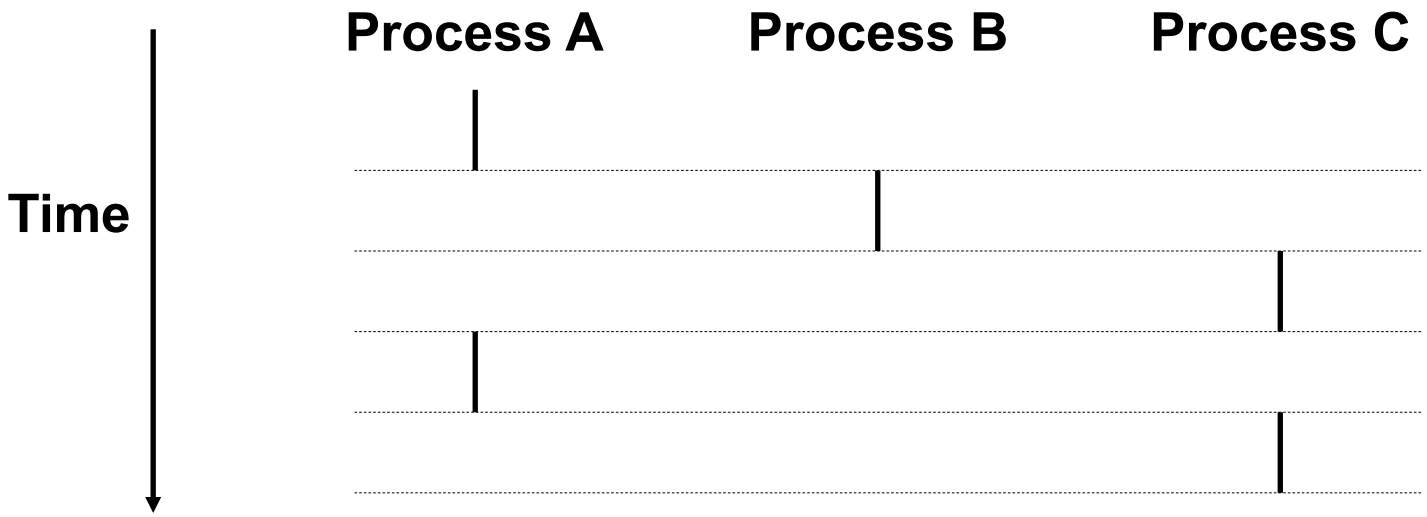
Context switching

- The scheduler (a chunk of kernel code) does scheduling as following
  - Decides whether to preempt the current process during the execution of a process
  - Selects a previously preempted process (scheduled process) to restart
  - Preempts the current process
    - Saves the context of the current process
  - Restarts the scheduled process
    - Restores the saved context of the scheduled process
    - Passes the control to this newly restored process

Concurrency

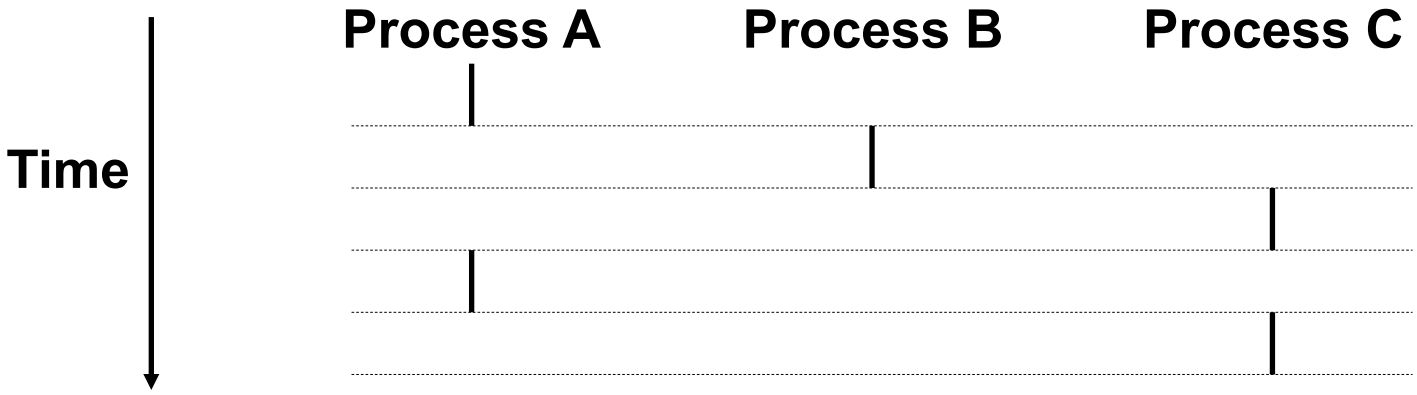
Logical control flows

- Each process has its own logical control flow
  - does not affect the states of any other processes
- Preempted
- Time slice



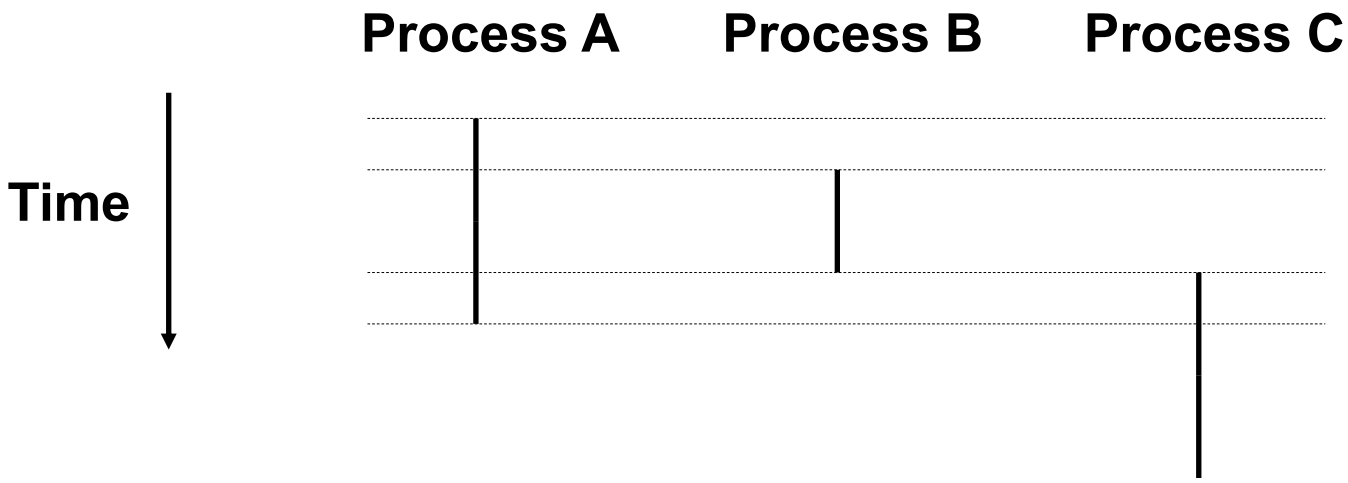
Concurrent processes

- Two processes run concurrently
  - are concurrent, if their flows overlap in time
- Otherwise, they are sequential
- Examples:
  - Concurrent: A & B, A & C
  - Sequential: B & C



User view of concurrent processes

- Control flows for concurrent processes are physically disjoint in time
- However, we can think of concurrent processes are running in parallel with each other logically
- Multitasking or time slicing



Three states of a process

- Running
  - the process is either executing on the CPU
  - or is waiting to be executed
  - and will eventually be scheduled
- Stopped (blocked)
  - the execution of the process is suspended and will not be scheduled
- Terminated
  - the process is stopped permanently

Three states of a process

- A running process becomes stop
  - by receiving a signal such as SIGSTOP
- A stopped process becomes running
  - by receiving a SIGCONT signal
- A process becomes terminated
  - receiving a signal whose default action is to terminate the process
  - returning from the main routine (calling the exit function)

System Call Error Handling

System Call Error Handling

- Unix system-level functions encounter an error
  - typically return –1
  - set the global integer variable **`errno`**
    - to indicate what went wrong

System Call Error Handling

```
1  if ((pid = fork()) < 0) {
2       fprintf(stderr, "fork error: %s\n",
3               strerror(errno));
3       exit(0);
4  }
```

```
1 void unix_error(char *msg) /* unix-style error */
2 {
3       fprintf(stderr, "%s: %s\n", msg, strerror(errno));
4       exit(0);
5 }
```

System Call Error Handling

```
1 pid_t Fork(void)
2 {
3     pid_t pid;
4
5     if ((pid = fork()) < 0)
6           unix_error("Fork error");
7     return pid;
8 }
```

Create and Terminate a Process

Obtaining Process ID's

- **`Process ID`**
  - PID
  - Each process has a unique positive PID

- **`Getpid()`**
  - Returns the PID of the calling process
- **`Getppid()`**
  - Returns the PID of its parent
    - The process that created the calling process

Obtaining Process ID's

```
#include <unistd.h>
#include <sys/types.h>
pid_t getpid(void);
pid_t getppid(void);
```

returns: PID of either the caller or the parent

- The **getpid** and **getppid** routines return an integer value of type **pid_t**
- **pid_t**
  - Defined in **types.h** as an **int** on Linux systems

Exit Function

```
#include <stdlib.h>
void exit(int status);
```

this function does not return

- The **exit** function terminates the process with an **exit status** of status

Fork Function

- A parent process
  - creates a new running child process
  - by calling the **`fork`** function

```
#include <unistd.h>
#include <sys/types.h>
pid_t fork(void);
```

Returns: 0 to child, PID of child to parent, -1 on error

Fork Function

- The newly created child process is
  - almost, but not quite, identical to the parent
- The parent and the newly created child have different PIDs

Fork Function

- The child
  - gets an identical (but separate) copy of the parent's user-level virtual address space
    - including the text, data, and bss segments, heap, and user stack
  - also gets identical copies of any of the parent's open file descriptors
    - which means the child can read and write any files that were open in the parent when it called **`fork`**

Fork Function

- Called once
  - In the parent process
- Returns twice:
  - In the parent process
    - Return the PID of the child
  - In the newly created child process
    - Return 0

Fork Function

- The return value provides an unambiguous way
  - whether the program is executing in the parent or the child

Fork Function

```
1 #include "csapp.h"
2
3 int main()
4 {
5       pid_t pid;
6       int x = 1;
7
8       pid = Fork();
9       if (pid == 0) { /* child */
10              printf("child : x=%d\n", ++x);
11              exit(0);
12      }
13
14      /* parent */
15      printf("parent: x=%d\n", --x);
16      exit(0);
17 }
```

Fork Function

- Call once, return twice
  - As mentioned before
  - This is fairly straightforward for programs that create a single child
  - Programs with multiple instances of `fork`
    - can be confusing
    - need to be reasoned about carefully

Fork Function

- Concurrent execution
  - The parent and the child are separate processes that run concurrently
  - The instructions in their logical control flows can be interleaved by the kernel in an arbitrary way
  - We can never make assumptions about the interleaving of the instructions in different processes

Fork Function

- Duplicate but separate address spaces
  - Immediately after the **`fork`** function returned in each process, the address space of each process is identical
    - Local variable **`x`** has a value of 1 in both the parent and the child when the **`fork`** function returns in line 8

Fork Function

```
1 #include "csapp.h"
2
3 int main()
4 {
5       pid_t pid;
6       int x = 1;
7
8       pid = Fork();
9       if (pid == 0) { /* child */
10              printf("child : x=%d\n", ++x);
11              exit(0);
12      }
13
14      /* parent */
15      printf("parent: x=%d\n", --x);
16      exit(0);
17 }
```

Fork Function

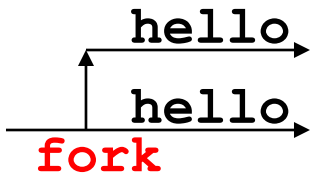
- Duplicate but separate address spaces.
  - The parent and the child are separate processes
    - they each have their own private address spaces.
    - Any subsequent changes that a parent or child makes to **x**
      - private
      - not reflected in the memory of the other process.

Fork Function

- Duplicate but separate address spaces
  - The variable **`x`** has different values in the parent and child when they call their respective **`printf`** statements
- Shared files
  - Like **`stdout`**
  - Communication between child and parent

Fork Function

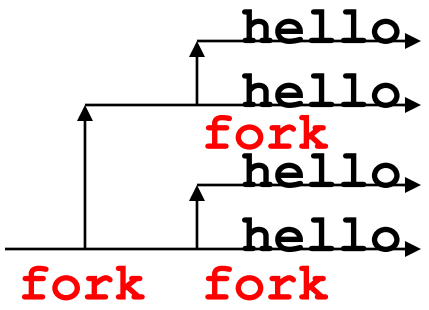
```
1 #include "csapp.h"
2
3 int main()
4 {
5       Fork();
6       printf("hello!\n");
7       exit(0);
8 }
```



(a) Calls fork once. (b) Prints two output lines.

Fork Function

```
1 #include "csapp.h"
2
3 int main()
4 {
5       Fork();
6       Fork();
7       printf("hello!\n");
8       exit(0);
9 }
```

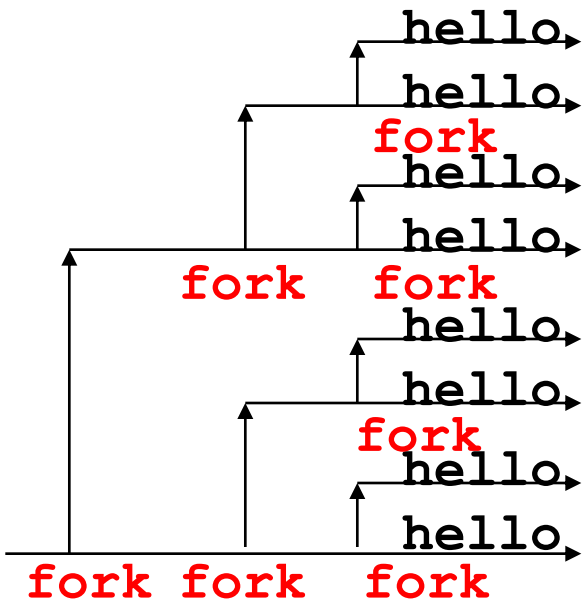


(c) Calls fork twice.

(d) Prints four output lines.

Fork Function

```
1 #include "csapp.h"
2
3 int main()
4 {
5       Fork();
6       Fork();
7       Fork();
8       printf("hello!\n");
9       exit(0);
10 }
```



(e) Calls fork three times. (f) Prints eight output lines.